Bias Amplification Enhances Minority Group Performance

Anonymous Authors[1]

Abstract

Neural networks produced by standard training are known to suffer from poor accuracy on rare subgroups despite achieving high accuracy on average, due to the correlations between certain spurious features and labels. Previous approaches based on worst-group loss minimization (*e.g.* Group-DRO) are effective in improving worse-group accuracy but require expensive group annotations for all the training samples. In this paper, we focus on the more challenging and realistic setting where group annotations are only available on a small validation set or are not available at all. We propose BAM, a novel two-stage training algorithm: in the first stage, the model is trained using a *bias amplification* scheme via introducing a learnable *auxiliary variable* for each training sample; in the second stage, we upweight the samples that the bias-amplified model misclassifies, and then continue training the same model on the reweighted dataset. Empirically, BAM leads to consistent improvement over its counterparts in worst-group accuracy, resulting in state-of-the-art performance in spurious correlation benchmarks in computer vision and natural language processing. Moreover, we find a simple stopping criterion based on *minimum class accuracy difference* that can completely remove the need for group annotations, with little or no loss in worst-group accuracy. We perform extensive analyses and ablations to verify the effectiveness and robustness of our algorithm in varying class and group imbalance ratios.

1. Introduction

The presence of spurious correlations in the data distribution, also referred to as "shortcuts" (Geirhos et al., 2020), is known to cause machine learning models to generate unintended decision rules that rely on spurious features. For example, image classifiers can largely use background instead of the intended combination of object features to make predictions (Beery et al., 2018).

Similar phenomenon is also prevalent in natural language processing (Gururangan et al., 2018) and reinforcement learning (Lehman et al., 2020). In this paper, we focus on the *group robustness* formulation of such problems (Sagawa et al., 2019), where we assume the existence of *spurious attributes* in the training data and define *groups* to be the combination of class labels and spurious attributes. The objective is to achieve high *worst-group accuracy* on test data, which would indicate that the model is not exploiting the spurious attributes.

Under this setup, one type of methods use a distributionally robust optimization framework to directly minimize the worst-group training loss (Sagawa et al., 2019). While these methods are effective in improving worst-group accuracy, they require knowing the group annotations for all training examples, which is expensive and oftentimes unrealistic. In order to resolve this issue, a line of recent work focused on designing methods that do not require group annotations for the training data, but need them for a small set of validation data (Liu et al., 2021; Nam et al., 2020; 2022; Zhang et al., 2022). A common feature shared by these methods is that they all consist of training two models: the first model is trained using plain empirical risk minimization (ERM) and is intended to be "biased" toward certain groups; then, certain results from the first model are utilized to train a debiased second model to achieve better worst-group performance. For instance, a representative method is JTT (Liu et al., 2021), which, after training the first model using ERM for a few epochs, trains the second model while upweighting the training examples incorrectly classified by the first model.

The core question that motivates this paper is: *Since the first model is intended to be biased, can we amplify its bias in order to improve the final group robustness?* Intuitively, a bias-amplified first model can provide better information to guide the second model to be debiased, which can potentially lead to improving group robustness. To this end, we propose a novel two-stage algorithm, BAM (Bias AMplifi-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

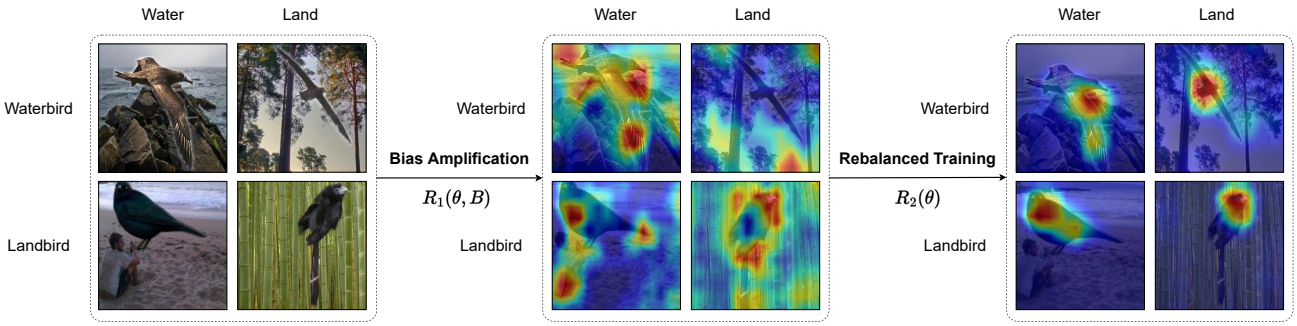

Figure 1. Using Grad-CAM (Selvaraju et al., 2017) to visualize the effect of bias amplification and rebalanced training stages, where the classifier heavily relies on the background information to make predictions after bias amplification but focuses on the useful feature (bird) itself after the rebalanced training stage.

cation), for improving worst-group accuracy without any group annotations for training data:

- *Stage 1: Bias amplification.* We train a bias-amplified model by introducing a trainable auxiliary variable for each training example.
- *Stage 2: Rebalanced training.* We upweight the training examples that are misclassified in Stage 1, and continue training the same model instead of retraining a new model.[1]

Evaluated on various standard benchmark datasets for spurious correlations, including Waterbirds (Wah et al., 2011; Sagawa et al., 2019), CelebA (Liu et al., 2015; Sagawa et al., 2019), MultiNLI (Williams et al., 2018; Sagawa et al., 2019), and CivilComments-WILDS (Borkan et al., 2019; Koh et al., 2021), we find that BAM achieves state-of-the-art worst-group accuracy compared to existing methods in the setting where group annotations are only available on a validation set.

Digging deeper into the mechanism of BAM, we observe that auxiliary variables learned in Stage 1 exhibit clearly different magnitudes between majority and minority group examples, thus confirming their effectiveness in bias amplification. We also find that BAM achieves robust performance across hyperparameter choices. In addition, our ablation studies demonstrate the clear advantage of continued training in Stage 2 over training a separate model from scratch.

Furthermore, we explore the possibility of *completely* removing the need for group annotations. We find that low class accuracy difference (which does not require any group annotations to evaluate) is strongly correlated with high worst-group accuracy. Using minimum class accuracy difference as the stopping criterion, BAM outperforms the previous state-of-the-art annotation-free method, GEORGE (Sohoni et al., 2020), by a considerable margin, and closes the performance gap between GEORGE and fully-supervised Group-DRO by an average of 88% on the image classification datasets. We also perform extensive controlled experiments and find that this approach is robust across different datasets, varying dataset sizes, and varying imbalance ratios.

[1] In Figure 1, we use Grad-CAM visualization to illustrate that our bias-amplified model from Stage 1 focuses more on the image background while the final model after Stage 2 focuses on the object target.

2. Related Works

A variety of recent work discussed different realms of robustness, for instance, class imbalance (He & Garcia, 2009; Huang et al., 2016; Khan et al., 2017; Johnson & Khoshgoftaar, 2019; Thabtah et al., 2020), and robustness in distribution shift, where the target data distribution is different from the source data distribution (Clark et al., 2019; Zhang et al., 2020; Marklund et al., 2020; Lee et al., 2022; Yao et al., 2022). In this paper, we mainly focus on improving group robustness. Categorized by the amount of information available for training and validation, we discuss three directions below.

Improving Group Robustness with Training Group Annotations. Multiple works used training group annotations to improve worst-group accuracy (Byrd & Lipton, 2019; Khani et al., 2019; Goel et al., 2020; Cao et al., 2020; Sagawa et al., 2020). Other works include minimizing the worst-group training loss using distributionally robust optimization (Group-DRO) (Sagawa et al., 2019), simple training data balancing (SUBG) (Idrissi et al., 2022), and retraining the last layer of the model on the group-balanced dataset (DFR) (Kirichenko et al., 2022). These methods achieve state-of-the-art performance on all benchmark datasets. However, the acquisition of spurious attributes of the entire training set is extremely expensive and often unrealistic.

Improving Group Robustness with Validation Group Annotations Only. Acknowledging the cost of obtaining group annotations, many recent works focus on the setting where training group annotations are not available (Duchi & Namkoong, 2019; Oren et al., 2019; Levy et al., 2020; Pezeshki et al., 2021). Taghanaki et al. (2021) proposed a transformation network to remove the spurious correlated features from image datasets and then choose classifier architectures according to the downstream task. Shu et al. (2019) utilized a small set of unbiased meta-data to reweight data samples. CVaR DRO (Duchi et al., 2019) introduced a variant of distributionally robust optimization that dynamically reweights data samples that have the highest losses. In particular, the most popular recent methods of achieving high worst-group accuracy involve training two models. CNC (Zhang et al., 2022) first trains an ERM model to help infer pseudo-group labels by clustering output features and then adopts a standard contrastive learning approach to improve robustness. SSA (Nam et al., 2022) uses a subset of the validation samples with group annotations for training to obtain pseudo spurious attributes, and then trains a robust model by minimizing worst-group loss, the same as Group-DRO. Similarly, $\text{DFR}^{\text{Val}}_{\text{Tr}}$ (Kirichenko et al., 2022) uses validation data with group annotations for training and tuning hyperparameters, though it just requires retraining the last layer of the model. Kim et al. (2022) introduces a multi-model approach that identifies hard-to-learn samples and obtains their weights based on the consensus of member classifiers of a committee, and simultaneously trains a main classifier through knowledge distillation.

Most related to this paper are the approaches that use one model to identify minority samples and then train a second model based on the results predicted by the first model (Yaghoobzadeh et al., 2019; Utama et al., 2020). LfF (Nam et al., 2020) trains two models concurrently, where one model is intentionally biased, and the other one is debiased by reweighting the gradient of the loss according to a relative difficulty score. JTT (Liu et al., 2021) first trains an ERM model to identify minority groups in the training set (similar to EIIL (Creager et al., 2021)), and then trains a second ERM model with these selected samples to be upweighted. Our algorithm improves over these methods by introducing trainable per-example auxiliary variables for bias amplification in the first model, as well as using continued training instead of training a separate second model.

Improving Group Robustness without any Group Annotations. Relatively less work has been done under the condition that no group information is provided for either training or validation. Idrissi et al. (2022); Liu et al. (2021) observed a significant drop (10% - 25%) in worst-group test accuracy if using the highest *average* validation accuracy as the stopping criterion without any group annotations. GEORGE (Sohoni et al., 2020) generates pseudo group annotations by performing clustering in the neural network's feature space, and uses the pseudo annotations to train a robust model via distributionally robust optimization. Seo et al. (2022) clustered the pseudo-attributes based on the embedding feature of a naively trained model, and then defined a trainable factor to reweight different clusters based on their sizes and target losses. However, there is a considerable performance gap between annotation-free methods and those that use group annotations.

The method of per-example auxiliary variables was first introduced in Hu et al. (2020) for a different motivation (learning with noisy labels). Hu et al. (2020) proved that it recovers ℓ_2 regularization when the neural network is in the Neural Tangent Kernel (Jacot et al., 2018) regime, and provided a theoretical guarantee for the noisy label learning problem. Our result shows that this method is also applicable to improving worst-group accuracy through its bias amplification effect.

3. Preliminaries

We adopt the group robustness formulation for spurious correlation (Sagawa et al., 2019). Consider a classification problem where each sample consists of an input $x \in \mathcal{X}$, a label $y \in \mathcal{Y}$, and a spurious attribute $a \in \mathcal{A}$. For example, in CelebA, $\mathcal{X}$ contains images of human faces and we want to classify hair color into the labels $\mathcal{Y} = \{\text{blonde}, \text{not blonde}\}$. Hair color may be highly correlated with gender $\mathcal{A} = \{\text{male}, \text{female}\}$ which is a spurious feature that can also predict the label. We say that each example belongs to a group $g = (y, a) \in \mathcal{G} = \mathcal{Y} \times \mathcal{A}$.

Let $f : \mathcal{X} \to \mathcal{Y}$ be a classifier learned from a training dataset $D = \{(x_i, y_i)\}_{i=1}^n$. We hope that f does not overly rely on the spurious feature a to predict the label. To this end, we evaluate the model through its *worst-group error*:

$$\text{Err}_{\text{wg}}(f) := \max_{g \in \mathcal{G}} \mathbb{E}_{x,y|g}[\mathbb{1}[f(x) \neq y]].$$

We focus on the setting where no group annotations are available in the training dataset. We consider two cases under this setting: (1) group annotations are available in a validation set solely for the purpose of hyperparameter tuning, and (2) no group annotations are available at all. We will distinguish between these cases when comparing them with existing methods.

4. Our Approach: BAM

We now present BAM, a two-stage approach to improving worst-group accuracy without any group annotations at training time. In Stage 1, we train a *bias-amplified model* and select examples that this model makes mistakes on. Then, in Stage 2, we continue to train the same model while upweighting the samples selected from Stage 1.

Algorithm 1 BAM

input Training dataset D, number of epochs T in Stage 1, auxiliary coefficient λ, and upweight factor μ.

Stage 1: Bias Amplification

1. Optimize $R_1(\theta, B)$ (1) for T epochs and save the model parameters $\hat{\theta}_{\text{bias}}$.
2. Construct the error set E (2) misclassified by $\hat{f}_{\text{bias}}(\cdot) = f_{\hat{\theta}_{\text{bias}}}(\cdot)$.

Stage 2: Rebalanced Training

3. Continue training the model starting from $\hat{\theta}_{\text{bias}}$ to optimize $R_2(\theta)$ (3).
4. Apply a stopping criterion:
   - If group annotations are available for validation, stop at the highest worst-group validation accuracy;
   - If no group annotations are available, stop at the lowest validation class difference (4).

4.1. Stage 1: Bias Amplification

The key intuition behind previous two-stage approaches (*e.g.* JTT) is that standard training via ERM tends to first fit easy-to-learn groups with spurious correlations, but not the other hard-to-learn groups where spurious correlations are not present. Therefore, the samples that the model misclassified in the first stage can be treated as a proxy for hard-to-learn groups and used to guide the second stage.

We design a bias-amplifying scheme in Stage 1 with the aim of identifying a higher-quality error set to guide training in Stage 2. In particular, we introduce a trainable auxiliary variable for each example and add it to the output of the network. Let $f_\theta : \mathcal{X} \to \mathbb{R}^C$ be the neural network with parameters θ, where $C = |\mathcal{Y}|$ is the total number of classes. We use the following objective function in Stage 1:

$$R_1(\theta, B) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i) + \lambda b_i, y_i). \tag{1}$$

Here, $b_i \in \mathbb{R}^C$ is the auxiliary variable for the i-th example in the training set, and the collection of auxiliary variables $B = (b_1, \ldots, b_n)$ is learnable and is learned together with the network parameters θ via gradient-based optimization (B is initialized to be all 0). ℓ is the loss function. λ is a hyperparameter that controls the strength of the auxiliary variables: if $\lambda = 0$, this reduces to standard ERM and auxiliary variables are not used; the larger λ is, the more we are relying on auxiliary variables to reduce the loss.

The introduction of auxiliary variables makes it more difficult for the network f_θ to learn, because the auxiliary variables can do the job of fitting the labels. We expect this effect to be more pronounced for hard-to-learn examples. For example, if in normal ERM training it takes a long time for the network f_θ to fit a hard-to-learn example (x_i, y_i), after introducing the auxiliary variable b_i, it will be much easier to use b_i to fit the label y_i, thus making the loss $\ell(f_\theta(x_i) + \lambda b_i, y_i)$ drop relatively faster. This will prohibit the network f_θ itself from learning this example. Such effect will be smaller for easy-to-learn examples, since the network itself can still quickly fit the labels without much reliance on the auxiliary variables. Therefore, **adding auxiliary variables amplifies the bias toward easy-to-learn examples**, making hard examples even harder to learn.

In terms of the computational cost, our method can be scaled to very large datasets even though it requires an auxiliary variable per example. In fact, we can store the auxiliary variables on the disk instead of in memory. During training, we only load the auxiliary variables corresponding to the current mini-batch, update these variables and save their new values on the disk. In this way, we only incur little additional memory and computation compared to plain ERM.

At the end of Stage 1, we evaluate the obtained model $\hat{f}_{\text{bias}}(\cdot) = f_{\hat{\theta}_{\text{bias}}}(\cdot)$ on the training set and identify an *error set*: (note that auxiliary variables are now removed)

$$E = \{(x_i, y_i) \colon \hat{f}_{\text{bias}}(x_i) \neq y_i\}. \tag{2}$$

4.2. Stage 2: Rebalanced Training

In Stage 2, we continue training the model starting from the parameters $\hat{\theta}_{\text{bias}}$ from Stage 1, using a rebalanced loss that upweights the examples in the error set E:

$$R_2(\theta) = \mu \sum_{(x,y)\in E} \ell(f_\theta(x), y) + \sum_{(x,y)\in D\setminus E} \ell(f_\theta(x), y), \tag{3}$$

where μ is a hyperparamter (upweight factor). For fair comparison, our implementation of upweighting follows that of JTT (Liu et al., 2021), i.e., we construct an upsampled dataset containing examples in the error set μ times and all other examples once.

We note that more complicated approaches have been proposed for Stage 2, *e.g.* Zhang et al. (2022), but we stick with the simple rebalanced training method in order to focus on the bias amplification effect in Stage 1.

4.3. Stopping Criterion without Any Group Annotations – Class Difference

When group annotations are available in a validation set, we can simply use the *worst-group validation accuracy* as a stopping criterion and to tune hyperparameters, similar to prior approaches (Nam et al., 2020; Liu et al., 2021; Creager et al., 2021). When no group annotations are available, a naive approach is to use the validation average accuracy as a proxy, but this results in poor worst-group accuracy (Liu et al., 2021; Idrissi et al., 2022).

Table 1. Average and worst-group test accuracies of different approaches evaluated on image datasets (Waterbird and CelebA). We run BAM and JTT (in *) on 3 random seeds based on the highest worst-group validation accuracies and minimum class differences, respectively, and report the mean and standard deviation. Results of EIIL and Group-DRO are reported by (Nam et al., 2022), and results of other approaches come from their original papers. The best worst-group accuracies under the same condition are marked in **bold**.

Annotation free?	Annotation only used for tuning h-params?	Method	Waterbird		CelebA	
			Avg.	Worst-group	Avg.	Worst-group
No	No	SUBG (Idrissi et al., 2022)	-	89.1$\pm$1.1	-	85.6$\pm$2.3
		GroupDRO (Sagawa et al., 2019)	91.8$\pm$0.48	**89.2**$\pm$0.18	93.1$\pm$0.21	88.5$\pm$1.16
		SSA (Nam et al., 2022)	92.2$\pm$0.87	89.0$\pm$0.55	92.8$\pm$0.11	**89.8**$\pm$1.28
	Yes	ERM	97.3	72.6	95.6	47.2
		EIIL (Creager et al., 2021)	96.9	78.7	91.9	83.3
		CNC (Zhang et al., 2022)	90.9$\pm$0.1	88.5$\pm$0.3	89.9$\pm$0.5	**88.8**$\pm$0.9
		JTT*	89.9$\pm$0.41	86.8$\pm$1.61	91.3$\pm$0.36	78.7$\pm$1.15
		BAM	91.4$\pm$0.44	**89.7**$\pm$0.26	88.0$\pm$0.37	83.5$\pm$0.94
Yes	-	GEORGE (Sohoni et al., 2020)	95.7	76.2	94.8	52.4
		JTT + ClassDiff*	88.5$\pm$1.47	87.1$\pm$0.24	91.8$\pm$0.76	75.4$\pm$3.28
		BAM + ClassDiff	91.4$\pm$0.31	**89.2**$\pm$0.15	88.4$\pm$2.32	**80.3**$\pm$3.32

Table 2. Average and worst-group test accuracies of different approaches evaluated on natural language datasets (MultiNIL and CivilComments-WILDS), following the same conventions in Table 1.

Annotation free?	Annotation only used for tuning h-params?	Method	MultiNLI		CivilComments-WILDS	
			Avg.	Worst-group	Avg.	Worst-group
No	No	SUBG (Idrissi et al., 2022)	-	68.9$\pm$0.8	-	**71.8**$\pm$0.4
		GroupDRO (Sagawa et al., 2019)	81.4$\pm$1.40	**76.6**$\pm$0.41	87.7$\pm$1.35	69.1$\pm$1.53
		SSA (Nam et al., 2022)	79.9$\pm$0.87	76.6$\pm$0.66	88.2$\pm$1.95	69.9$\pm$2.02
	Yes	ERM	82.4	67.9	92.6	57.4
		EIIL (Creager et al., 2021)	79.4	70.9	90.5	67.0
		CNC (Zhang et al., 2022)	-	-	81.7$\pm$0.5	68.9$\pm$2.1
		JTT*	80.0$\pm$0.41	68.1$\pm$0.90	87.2$\pm$1.65	77.7$\pm$1.70
		BAM	79.6$\pm$1.11	**71.5**$\pm$1.56	88.3$\pm$ 0.76	**79.3**$\pm$2.69
Yes	-	GEORGE (Sohoni et al., 2020)	-	-	-	-
		JTT + ClassDiff*	81.2$\pm$0.56	66.5$\pm$0.56	87.2$\pm$1.65	77.7$\pm$1.70
		BAM + ClassDiff	80.3$\pm$0.99	**71.2**$\pm$1.52	88.3$\pm$0.76	**79.3**$\pm$2.69

We identify a simple heuristic when no group annotations are available, using *minimum class difference*, which we find to be highly effective and result in little or no loss in worst-group accuracy. For a classification problem with C classes, we calculate the sum of pairwise validation accuracy differences between classes as

$$\text{ClassDiff} = \sum_{i,j=1}^{C} |\text{Acc}(\text{class } i) - \text{Acc}(\text{class } j)|. \quad (4)$$

ClassDiff can be calculated on a validation set without any group annotations. The main intuition for ClassDiff is that we expect the worst-group accuracy to be (near) highest when all group accuracies have a relatively small gap between each other. A weaker condition for this to hold is that the class accuracies have small difference, which can be estimated by ClassDiff without access to any group annotations.

In all the datasets (where $C = 2$ or 3) we experiment with, as well as for varying dataset size, class imbalance ratio, and group imbalance ratio, we observe that ClassDiff inversely correlates with worst-group accuracy (see Section 6.3). Our

results suggest that ClassDiff is a useful stopping criterion when no group annotations are available.

Our algorithm is summarized in Algorithm 1. It has three hyperparameters: T (number of epochs in Stage 1), λ (auxiliary variable coefficient), and μ (upweight factor). We provide full training details in Appendix B.

5. Experiments

In this section, we demonstrate that BAM improves worst-group accuracy compared to prior methods that are trained without spurious attributes on standard benchmark datasets.

5.1. Setup

We conduct our experiments on four popular benchmark datasets containing spurious correlations. Two of them are image datasets: Waterbirds and CelebA, and the other two are NLP datasets: MultiNLI and CivilComments-WILDS. The full dataset details are in Appendix A. BAM is trained in the absence of training group annotations throughout all experiments. We obtain the main results of BAM via tuning with and without group annotations on the validation set, following Algorithm 1.

For a fair comparison, we adopt the general settings from previous methods (JTT) and stay consistent with other approaches without extensive hyperparameter tuning (batch size, learning rate, regularization strength in Stage 2, etc.). We use pretrained ResNet-50 (He et al., 2016) for image datasets, and pretrained BERT (Devlin et al., 2019) for NLP datasets. More details can be found in Appendix B.

5.2. Results

Tables 1 and 2 report the average and worst-group test accuracies of BAM and compare it against standard ERM and recently proposed methods under different conditions, including SUBG (Idrissi et al., 2022), JTT (Liu et al., 2021), SSA (Nam et al., 2022), CNC (Zhang et al., 2022), GEORGE (Sohoni et al., 2020), and Group-DRO (Sagawa et al., 2019). We tune BAM and JTT according to the highest worst-group validation accuracy (not annotation-free) and minimum class difference (annotation-free).

First, compared with other methods that use group annotations only for hyperparameter tuning, BAM consistently achieves higher worst-group accuracies across all datasets, with the exception of the CelebA dataset on which CNC achieves better performance. We note that CNC primarily focuses on improving Stage 2 with a more complicated contrastive learning method, while BAM uses the simple upweighting method. It is possible that the combination of CNC and BAM could lead to better results. Nevertheless, the result of BAM is promising on all other datasets, even surpassing the weakly-supervised method SSA and the fully-supervised method Group-DRO on Waterbirds and CivilComments-WILDS.

Second, if the validation group annotations are not available at all, BAM achieves state-of-the-art performance on all four benchmark datasets and surpasses previous methods by a large margin. Notably, BAM recovers a significant portion of the gap in worst-group accuracy between GEORGE (previous state-of-the-art that requires no group annotations) and Group-DRO/SSA (previous state-of-the-art requiring supervision) by an average of 88% on the image classification datasets.

BAM's improved performance in worst-group accuracy comes at the expense of a moderate drop in average accuracy. The tradeoff between average accuracy and worst-group accuracy is consistent with the observation made by Liu et al. (2021); Sagawa et al. (2019). We note that our implementation of JTT follows directly from its published code, and we obtain a much higher performance on the CivilComments-WILDS dataset than originally reported by Liu et al. (2021).

6. Analyses and Ablations

In this section, we perform detailed analyses and ablations to better understand various components of BAM, including auxiliary variables (Section 6.2), ClassDiff (Section 6.3), sensitivity of hyperparameters λ and T (Section 6.4), and the advantage of continued training in Stage 2 (Section 6.5).

6.1. Additional Controlled Experiments Setup

Despite the popularity of the benchmark datasets, they have significantly different major-versus-minor class/group ratios and uncommon learning patterns (Idrissi et al., 2022). For a more rigorous analysis of our proposed auxiliary variable and classDiff, we set up three additional controlled datasets where we adjust different class/group imbalance ratios, including Colored-MNIST, Controlled-Waterbirds, and CIFAR10-MNIST. See Appendix A for details.

6.2. Analysis of Auxiliary Variables

We verify the intuition behind the auxiliary variables by visualizing their learned values and comparing between majority and minority examples. Recall that we expect minority examples to rely more on auxiliary variables (see Section 4.1).

6.2.1. Visualization of Auxiliary Variables

In Figure 2, we visualize the distribution of the learned auxiliary variables on each group of the original Waterbirds dataset at $T = 20$. We have two observations. First, auxiliary variables for examples in majority and minority groups

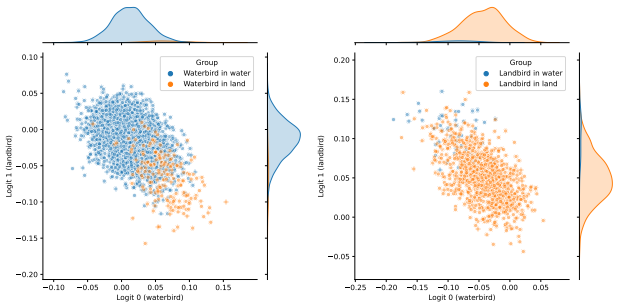

Figure 2. Distributions of the auxiliary variable w.r.t. the waterbird class (left) and landbird class (right) on the training set at $T = 20$. We use two distinct colors to illustrate the distributions of two groups in each class. The coordinates of data sample i relative to the origin show the bias learned by the auxiliary variable.

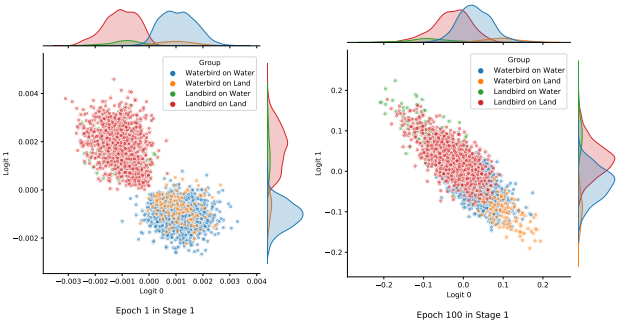

Figure 3. Epochs 1 and 100 in Stage 1 on Waterbirds. Logit 0 corresponds to the prediction on the waterbird class, and logit 1 corresponds to landbird. The group sizes are 1800, 200, 200, and 1800 in order.

have clear distinctions. Second, auxiliary variables for majority group examples are in general closer to the origin, while those in minority groups tend to have larger (positive) logit values on the ground truth class they actually belong to and have smaller (negative) logit values on the class they do not belong to. The visualization shows that the auxiliary variables help with fitting hard-to-learn samples, which corroborates the intuition described in Section 4.1. We observe the same trend for any $\lambda \in \{0.5, 5, 20, 50, 70, 100\}$ as well as any T in Stage 1 in $\{20, 50, 100\}$. This supports our intuition that "adding auxiliary variables amplifies the bias toward easy-to-learn examples" in Section 4.1.

6.2.2. Visualization as Training Progresses

To further explore if the auxiliary variables keep such trends in different scenarios, we take a deeper look into the training progression of the auxiliary variables in our controlled experiments, where we set a variety of different imbalance sizes/ratios. In Figures 3 and 4, scatter plots show how the values of auxiliary variables change as Stage 1 proceeds. Carefully controlling all other parameters and hyperparameters, we observe clear patterns on Controlled-Waterbirds and Colored-MNIST: First, as T grows, the logits become larger and increasingly influence the model predictions. Second, minority and majority groups are more separated from each other as training progresses, resulting in the bias amplification effect. Appendix C illustrates the values of b at more intermediate epochs T and clearly shows the trends.

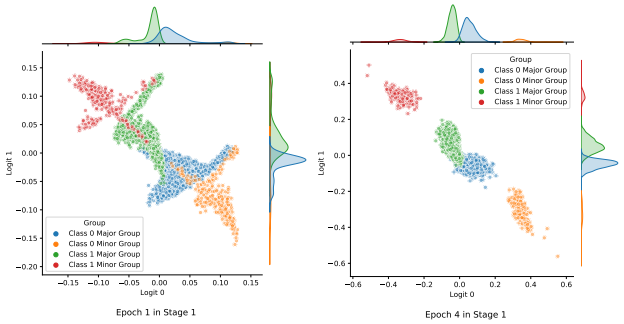

Figure 4. Epochs 1 and 4 in stage 1 on Colored-MNIST. Logit 0 corresponds to the prediction on class 0, and logit 1 corresponds to class 1.

6.3. Results of ClassDiff

We perform extensive experiments on the four benchmark datasets, as well as on our controlled datasets by altering class and group imbalance ratios. We find that ClassDiff works well on every single experiment, which shows the effectiveness and robustness of ClassDiff across different settings.

Real-World Benchmark Datasets Figure 5 plots the trend of the absolute class difference and the worst-group accuracy on the validation set in Stage 2 for the Waterbirds, CelebA, MultiNLI, and CiviComments. Clearly, there is a strong inverse relationship between the absolute class difference and worst-group accuracy in the validation set on all four datasets, which justifies the use of class difference as a stopping criterion.

Controlled Datasets On our controlled datasets (Colored-MNIST, CIFAR10-MNIST, and Controlled-Waterbirds), we vary the dataset sizes, class size ratios, and group size ratios. In Appendix D, we show the performance of ClassDiff in several randomly selected settings (we explain our procedure of selecting settings which does not involve cherry-picking). In *every single setting* we have tested, we observe a similar negative correlation as in Figure 5. These findings suggest that ClassDiff could be generally effective when no group annotation is available, as it is robust across different datasets, varying dataset sizes, and imbalance characteristics.

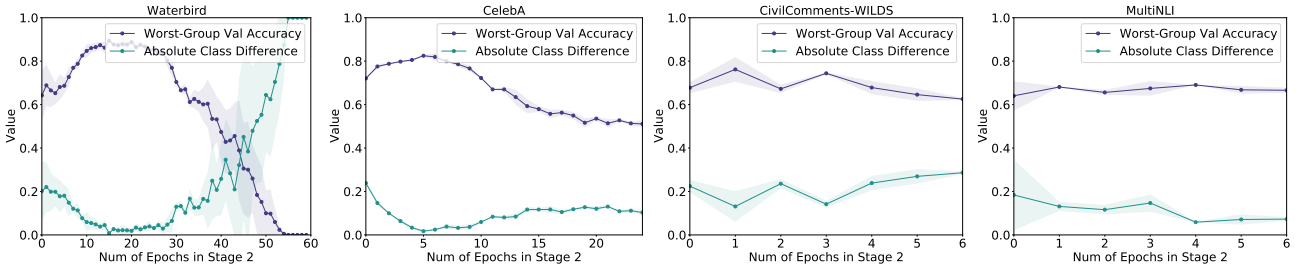

Figure 5. Relation between absolute valdiation class difference and worst-group validation accuracy in Stage 2 on Waterbirds, CelebA, CivilComments-WILDS, and MultiNLI. It can be observed that minimizing absolute validation class difference is roughly equivalent to maximizing worst-group accuracy in the validation set. Each dataset uses the same hyperparameters as employed in Table 1. Each line represents the value averaged over 3 different seeds and the shade represents the standard deviation.

6.4. Sensitivity of Hyperparameters

We find that BAM is robust to the choice of λ, as Table 3 below presents the best worst-group accuracies for a wide range of λ on Waterbirds.

Table 3. Sensitivity of λ

λ	0.5	10	30	35	40	50	70	100
Test acc	89.9	89.8	89.7	90.2	89.5	89.5	88.8	88.6

Furthermore, using a larger λ can eliminate the need to carefully tune the epoch number T in Stage 1. This resolves a major drawback of previous methods such as JTT, whose performance is sensitive with respect to the choice of T. Table 4 shows that even trained until full convergence in Stage 1, an appropriate choice of λ can still guarantee a state-of-the-art performance.

Table 4. Robustness of worst group test accuracy with different T's

λ	50	50	50	70	70	70
T	80	100	120	80	100	120
Test Acc	88.6	88.2	86.9	88.8	88.8	88.5

6.5. Ablation Studies on *One-M* and *Two-M*

We conduct ablation studies on the Waterbirds dataset to test the effectiveness of our proposed method of continued training in Stage 2. For consistent terminology, we define the approach that loads the model from Stage 1 and continues training the same model in Stage 2 as *One-M*. We define the approach that trains a separate ERM model in Stage 2 as *Two-M*. For a fair comparison, we employ the same hyperparameters throughout this section. We use the same stopping criterion here (highest worst-group validation accuracy). We tune over $T = \{10, 15, 20, 25, 30, 40, 50, 60, 80, 100, 120\}$ and $\mu = \{50, 100, 140\}$ for each setting, and fix $\lambda = 50$. To reduce the influence of randomness, we run all experiments using three different seeds and average the results.

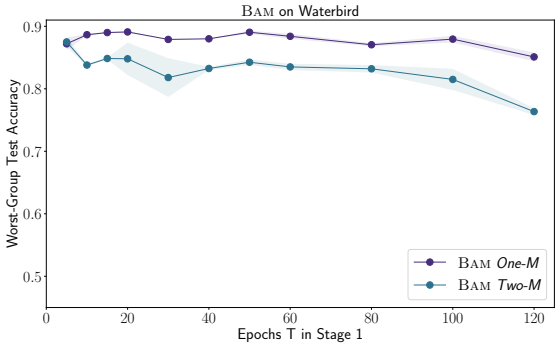

Figure 6. Comparison of BAM's performance between training *One-M* and *Two-M*. We find that *One-M* consistently achieves better worst-group test accuracies than *Two-M* for BAM over all hyperparameter combinations and BAM *One-M* demonstrates a stable performance regardless of the choice of T.

Figure 6 compares the performance between *One-M* and *Two-M* over a wide range of Stage 1 epochs T. Notably, *One-M* and *Two-M* share the same error sets in each T. The result suggests that *One-M* BAM outperforms *Two-M* BAM in every single Stage 1 epoch T, further verifying our intuition that the biased knowledge accumulated from Stage 1 helps with improving the final robustness. Additionally, as explained in Section 6.4 and also shown here in Figure 6, BAM demonstrates a stable performance, regardless of the choice of T's. By stark contrast, JTT suffers from significant performance degradation with large T's, as shown in Figure 3 of Liu et al. (2021).

7. Conclusion

We proposed BAM, a two-stage method that does not use any training group annotations and achieves state-of-the-art worst-group performance. We also introduced a novel stopping criterion, ClassDiff, that can remove the need for any group annotations.

References

Arjovsky, M., Bottou, L., Gulrajani, I., and Lopez-Paz, D. Invariant risk minimization. *arXiv preprint arXiv:1907.02893*, 2019.

Beery, S., Van Horn, G., and Perona, P. Recognition in terra incognita. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 456–473, 2018.

Borkan, D., Dixon, L., Sorensen, J., Thain, N., and Vasserman, L. Nuanced metrics for measuring unintended bias with real data for text classification. In *Companion proceedings of the 2019 world wide web conference*, pp. 491–500, 2019.

Byrd, J. and Lipton, Z. What is the effect of importance weighting in deep learning? In *International Conference on Machine Learning*, pp. 872–881. PMLR, 2019.

Cao, K., Chen, Y., Lu, J., Arechiga, N., Gaidon, A., and Ma, T. Heteroskedastic and imbalanced deep learning with adaptive regularization. *arXiv preprint arXiv:2006.15766*, 2020.

Cireşan, D. C., Meier, U., Masci, J., Gambardella, L. M., and Schmidhuber, J. High-performance neural networks for visual object classification. *arXiv preprint arXiv:1102.0183*, 2011.

Clark, C., Yatskar, M., and Zettlemoyer, L. Don't take the easy way out: Ensemble based methods for avoiding known dataset biases. *arXiv preprint arXiv:1909.03683*, 2019.

Creager, E., Jacobsen, J.-H., and Zemel, R. Environment inference for invariant learning. In *International Conference on Machine Learning*, pp. 2189–2200. PMLR, 2021.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL https://aclanthology.org/N19-1423.

Duchi, J. and Namkoong, H. Variance-based regularization with convex objectives. *The Journal of Machine Learning Research*, 20(1):2450–2504, 2019.

Duchi, J. C., Hashimoto, T., and Namkoong, H. Distributionally robust losses against mixture covariate shifts. *Under review*, 2, 2019.

Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., and Wichmann, F. A. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.

Goel, K., Gu, A., Li, Y., and Ré, C. Model patching: Closing the subgroup performance gap with data augmentation. *arXiv preprint arXiv:2008.06775*, 2020.

Gururangan, S., Swayamdipta, S., Levy, O., Schwartz, R., Bowman, S., and Smith, N. A. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pp. 107–112, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-2017. URL https://aclanthology.org/N18-2017.

He, H. and Garcia, E. A. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi: 10.1109/TKDE.2008.239.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016. doi: 10.1109/CVPR.2016.90.

Hu, W., Li, Z., and Yu, D. Simple and effective regularization methods for training on noisily labeled data with generalization guarantee. *arXiv: Learning*, 2020.

Huang, C., Li, Y., Loy, C. C., and Tang, X. Learning deep representation for imbalanced classification. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 5375–5384, 2016.

Idrissi, B. Y., Arjovsky, M., Pezeshki, M., and Lopez-Paz, D. Simple data balancing achieves competitive worst-group-accuracy. In *Conference on Causal Learning and Reasoning*, pp. 336–351. PMLR, 2022.

Jacot, A., Gabriel, F., and Hongler, C. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.

Johnson, J. M. and Khoshgoftaar, T. M. Survey on deep learning with class imbalance. *Journal of Big Data*, 6(1): 1–54, 2019.

Khan, S. H., Hayat, M., Bennamoun, M., Sohel, F. A., and Togneri, R. Cost-sensitive learning of deep feature representations from imbalanced data. *IEEE transactions on neural networks and learning systems*, 29(8):3573–3587, 2017.

Khani, F., Raghunathan, A., and Liang, P. Maximum weighted loss discrepancy. *arXiv preprint arXiv:1906.03518*, 2019.

Kim, N., Hwang, S., Ahn, S., Park, J., and Kwak, S. Learning debiased classifier with biased committee. *arXiv preprint arXiv:2206.10843*, 2022.

Kirichenko, P., Izmailov, P., and Wilson, A. G. Last layer re-training is sufficient for robustness to spurious correlations. *arXiv preprint arXiv:2204.02937*, 2022.

Koh, P. W., Sagawa, S., Marklund, H., Xie, S. M., Zhang, M., Balsubramani, A., Hu, W., Yasunaga, M., Phillips, R. L., Gao, I., Lee, T., David, E., Stavness, I., Guo, W., Earnshaw, B., Haque, I., Beery, S. M., Leskovec, J., Kundaje, A., Pierson, E., Levine, S., Finn, C., and Liang, P. Wilds: A benchmark of in-the-wild distribution shifts. In Meila, M. and Zhang, T. (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 5637–5664. PMLR, 18–24 Jul 2021. URL https://proceedings.mlr.press/v139/koh21a.html.

Krizhevsky, A., Hinton, G., et al. Learning multiple layers of features from tiny images. 2009.

Lee, Y., Yao, H., and Finn, C. Diversify and disambiguate: Learning from underspecified data. *arXiv preprint arXiv:2202.03418*, 2022.

Lehman, J., Clune, J., Misevic, D., Adami, C., Altenberg, L., Beaulieu, J., Bentley, P. J., Bernard, S., Beslon, G., Bryson, D. M., et al. The surprising creativity of digital evolution: A collection of anecdotes from the evolutionary computation and artificial life research communities. *Artificial life*, 26(2):274–306, 2020.

Levy, D., Carmon, Y., Duchi, J. C., and Sidford, A. Large-scale methods for distributionally robust optimization. *Advances in Neural Information Processing Systems*, 33: 8847–8860, 2020.

Liu, E. Z., Haghgoo, B., Chen, A. S., Raghunathan, A., Koh, P. W., Sagawa, S., Liang, P., and Finn, C. Just train twice: Improving group robustness without training group information. In Meila, M. and Zhang, T. (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 6781–6792. PMLR, 18–24 Jul 2021. URL https://proceedings.mlr.press/v139/liu21f.html.

Liu, Z., Luo, P., Wang, X., and Tang, X. Deep learning face attributes in the wild. *2015 IEEE International Conference on Computer Vision (ICCV)*, pp. 3730–3738, 2015.

Marklund, H., Xie, S. M., Zhang, M., Balsubramani, A., Hu, W., Yasunaga, M., Phillips, R. L., Beery, S., Leskovec, J., Kundaje, A., et al. Wilds: A benchmark of in-the-wild distribution shifts. *arXiv preprint arXiv:2012.07421*, 2020.

Nam, J., Cha, H., Ahn, S., Lee, J., and Shin, J. Learning from failure: De-biasing classifier from biased classifier. *Advances in Neural Information Processing Systems*, 33: 20673–20684, 2020.

Nam, J., Kim, J., Lee, J., and Shin, J. Spread spurious attribute: Improving worst-group accuracy with spurious attribute estimation. *arXiv preprint arXiv:2204.02070*, 2022.

Oren, Y., Sagawa, S., Hashimoto, T. B., and Liang, P. Distributionally robust language modeling. *arXiv preprint arXiv:1909.02060*, 2019.

Pezeshki, M., Kaba, O., Bengio, Y., Courville, A. C., Precup, D., and Lajoie, G. Gradient starvation: A learning proclivity in neural networks. *Advances in Neural Information Processing Systems*, 34:1256–1272, 2021.

Sagawa, S., Koh, P. W., Hashimoto, T. B., and Liang, P. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. *arXiv preprint arXiv:1911.08731*, 2019.

Sagawa, S., Raghunathan, A., Koh, P. W., and Liang, P. An investigation of why overparameterization exacerbates spurious correlations. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 8346–8356. PMLR, 13–18 Jul 2020. URL https://proceedings.mlr.press/v119/sagawa20a.html.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pp. 618–626, 2017.

Seo, S., Lee, J.-Y., and Han, B. Unsupervised learning of debiased representations with pseudo-attributes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16742–16751, 2022.

Shu, J., Xie, Q., Yi, L., Zhao, Q., Zhou, S., Xu, Z., and Meng, D. Meta-weight-net: Learning an explicit mapping for sample weighting. *Advances in neural information processing systems*, 32, 2019.

Sohoni, N., Dunnmon, J., Angus, G., Gu, A., and Ré, C. No subclass left behind: Fine-grained robustness in coarse-grained classification problems. *Advances in Neural Information Processing Systems*, 33:19339–19352, 2020.

Taghanaki, S. A., Choi, K., Khasahmadi, A. H., and Goyal, A. Robust representation learning via perceptual similarity metrics. In *International Conference on Machine Learning*, pp. 10043–10053. PMLR, 2021.

Thabtah, F., Hammoud, S., Kamalov, F., and Gonsalves, A. Data imbalance in classification: Experimental evaluation. *Information Sciences*, 513:429–441, 2020.

Utama, P. A., Moosavi, N. S., and Gurevych, I. Towards debiasing nlu models from unknown biases. *arXiv preprint arXiv:2009.12303*, 2020.

Wah, C., Branson, S., Welinder, P., Perona, P., and Belongie, S. J. The caltech-ucsd birds-200-2011 dataset. 2011.

Williams, A., Nangia, N., and Bowman, S. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 1112–1122, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-1101. URL https://aclanthology.org/N18-1101.

Yaghoobzadeh, Y., Mehri, S., Tachet, R., Hazen, T. J., and Sordoni, A. Increasing robustness to spurious correlations using forgettable examples. *arXiv preprint arXiv:1911.03861*, 2019.

Yao, H., Wang, Y., Li, S., Zhang, L., Liang, W., Zou, J., and Finn, C. Improving out-of-distribution robustness via selective augmentation. *arXiv preprint arXiv:2201.00299*, 2022.

Zhang, J., Menon, A., Veit, A., Bhojanapalli, S., Kumar, S., and Sra, S. Coping with label shift via distributionally robust optimisation. *arXiv preprint arXiv:2010.12230*, 2020.

Zhang, M., Sohoni, N. S., Zhang, H. R., Finn, C., and Ré, C. Correct-n-contrast: A contrastive approach for improving robustness to spurious correlations. *arXiv preprint arXiv:2203.01517*, 2022.

A. Dataset details

Waterbirds (Wah et al., 2011; Sagawa et al., 2019): Waterbirds is an image dataset consisting of two types of birds on different backgrounds. It contains images of birds directly cut and pasted on real-life images of different landscapes. We aim at classifying $\mathcal{Y} = \{\text{waterbird, landbird}\}$, which is spuriously correlated with the background $\mathcal{A} = \{\text{water, land}\}$. The train/validation/test splits is followed from Sagawa et al. (2019).

CelebA (Liu et al., 2015): CelebA is an image dataset consisting of celebrities with two types of hair color and different genders. we aim at classifying celebrities' hair color $\mathcal{Y} = \{\text{blond, not blond}\}$, which is spuriously correlated with the gender $\mathcal{A} = \{\text{male, female}\}$. The train/validation/test splits is followed from Sagawa et al. (2019).

MultiNLI (Williams et al., 2018): MultiNLI is a natural language processing dataset, where each sample consists of two sentences and the second sentence is entailed by, neutral with, or contradicts the first sentence, with the presence or absence of negation words. We aim at classifying the sentence relationship $\mathcal{Y} = \{\text{entailment, neutral, contradiction}\}$, which is spuriously correlated with the negation words $\mathcal{A} = \{\text{negation, no negation}\}$. The train/validation/test splits is followed from Sagawa et al. (2019).

CivilComments-WILDS (Borkan et al., 2019; Koh et al., 2021): CivilComments-WILDS is a natural language processing dataset consisting of online comment that is toxic or non-toxic, with or without the mentions of certain demographic identities (male, female, White, Black, LGBTQ, Muslim, Christian, and other religions). We aim at classifying whether the sentence is toxic $\mathcal{Y} = \{\text{toxic, non-toxic}\}$, which is spuriously correlated with the mentions of (demographic) identities $\mathcal{A} = \{\text{identity, no identity}\}$. The train/validation/test splits is followed from Koh et al. (2021).

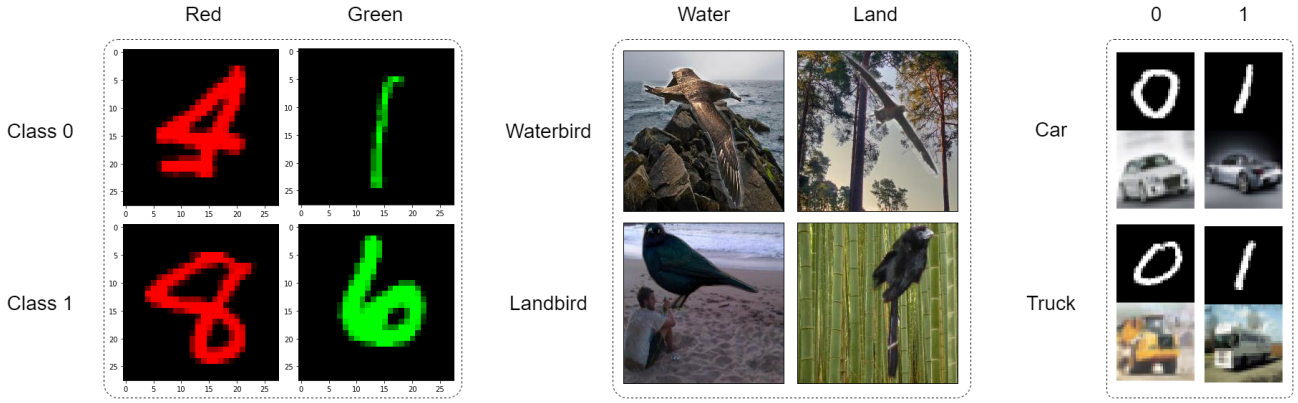

Figure 7. Examples of each group in Colored-MNIST, Controlled-Waterbirds, and CIFAR10-MNIST.

Colored-MNIST (Arjovsky et al., 2019; Cireşan et al., 2011): Colored-MNIST is an image dataset that colors each grayscale MNIST images either green or red and make it spuriously correlates with the digit values. We aim at classifying binary class labels $\mathcal{Y} = \{\text{class 0 (digits 0-4), class 1 (digits 5-9)}\}$ without relying on the color $\mathcal{A} = \{\text{red, green}\}$. Examples are shown in Figure 7. We shuffle the original data and generate dataset splits with train/valudation/test sizes $= 0.7/0.15/0.15$. In Figure 4, we set the total dataset size to be 10000, class imbalance ratio 1.0, and the group imbalance ratio within each class to be 0.1.

Controlled-Waterbirds (Wah et al., 2011; Sagawa et al., 2019): We manually construct this dataset from the original Waterbirds benchmark. We follow the previous definition that sets Waterrbird-Water and Landbird-Land as majority groups. We regenerate the dataset splits as train/valudation/test sizes $= 0.7/0.15/0.15$. Examples are shown in Figure 7. To show how the ClassDiff performs in various settings in Appendix D, we consider the following three circumstances: (1) We vary the group imbalance ratios, while the two classes are balanced. Due to the original dataset size limits, we fix the size of two majority groups to 1800, and vary the minority group sizes in $\{200, 400, 600, 800\}$. (2) The two classes are imbalanced. We fix sizes of the two groups in the smaller class as 200 and 1400, and choose the class imbalance ratio waterbird landbird in $\{1.0, 2.0, 3.0\}$. (3) Both group and class sizes are imbalanced. We consider a random setting of four group sizes, where Waterbird-Water : Waterbird-Land : Landbird-Water : Landbird-Land $= 1800 : 600 : 200 : 3600$.

CIFAR10-MNIST (Krizhevsky et al., 2009; Cireşan et al., 2011): We manually construct this dataset by concatenating

MNIST images onto CIFAR-10 images. We aim at classifying binary class labels $\mathcal{Y} = \{\text{car, truck}\}$, which is spuriously correlated with the MNIST digits $\mathcal{A} = \{0, 1\}$. We shuffle the original data and generate dataset splits with train/valudation/test sizes $= 0.7/0.15/0.15$. Examples are shown in Figure 7. To show how the ClassDiff performs in various settings in Appendix D, we consider the following two circumstances: (1) We vary the group imbalance ratios, while the two classes are balanced. Due to the original dataset size limits, we fix the size of two majority groups to 4000, and vary the minority group sizes in $\{400, 600, 800\}$. (2) The two classes are imbalanced. We fix sizes of the two groups in the larger class as 800 and 4000, and choose the class imbalance ratio car truck in $\{4/3, 4/2, 4/1\}$.

B. Training details

In this section, we provide details about the model selection and the hyperparameter tuning for different datasets. As claimed in the main text, in order to make a fair comparison with previous methods, we use the same pretrained models. Namely, we use ResNet-50 pretrained from Image-net weights for Waterbirds and CelebA, and pretrained BERT for MultiNLI and CivilComments. We use the `Pytorch` implementation for ResNet50 and the `HuggingFace` implementation for BERT. We tune BAM and JTT according to class difference and worst-group accuracies in the validation set in Stage 2.

Table 5. Hyperparameters tuned over 4 datasets.

Dataset	Auxiliary coefficient (λ)	#Epochs in Stage 1 (T)	Upweight factor (μ)
Waterbirds	{ 0.5, 5, 50}	{10, 15, 20}	{50, 100, 140}
CelebA	{0.5, 5, 50}	{1, 2}	{50, 70, 100}
MultiNLI	{0.5, 5, 50}	{1, 2}	{4, 5, 6}
CivilComments	{0.5, 5, 50 }	{1, 2}	{4, 5, 6}

In general, our setting follows closely from Liu et al. (2021), with some minor discrepancies. For the major hyperparameters, We tuned over $\lambda = \{0.5, 5, 50\}$, $T = \{1, 2, 10, 15, 60\}$ and $\mu = \{4, 5, 6, 50, 70, 100, 140\}$ for BAM. We note that BAM is fairly insensitive with the choice of λ as mentioned in Section 6.4. We tune over $T = \{1, 2\}$ and $\mu = \{4, 5, 6, 50, 70, 100, 140\}$ for JTT for fair comparisons. We tuned the major hyperparameters according to Table 5. More details are provided below:

Waterbirds We use the learning rate 1e-5 and batch size 64 for two stages of training. We use the stochastic gradient descent (SGD) optimizer with momentum 0.9 throughout the training process. We use ℓ_2 regularization 1 for Stage 2 *rebalanced training*. We apply the same above setting for both JTT and BAM. Notably, as illustrated by Figure 5, when tuned for the minimum absolute validation class difference, the curve may fluctuate abnormally after the first 30 epochs and it is clear that the model is not learning anything useful. We tackle this problem by smoothing out abrupt changes in class difference (neglect the result if the difference between consecutive class differences is greater than 10%). The best result for JTT occurs when $T = 60$ and $\mu = 140$. The best result for BAM occurs when $T = 20$ and $\mu = 140$. We train for a total of 360 epochs

CelebA We use the learning rate 1e-5 and batch size 128 for two stages of training. We use the stochastic gradient descent (SGD) optimizer with momentum 0.9 throughout the training process. We use ℓ_2 regularization 0.1 for Stage 2 *rebalanced training*. We apply the same above setting for both JTT and BAM. The best result for JTT occurs when $T = 1$ and $\mu = 70$. The best result for BAM occurs when $T = 1$ and $\mu = 50$. We train for a total of 60 epochs.

MultiNLI We use batch size 32 for two stages of training. We apply an initial learning rate of 2e-5 for Stage 1 and 1e-5 for Stage 2. We use the SGD optimizer without clipping for Stage 1 and the AdamW optimizer with clipping for Stage 2. We use ℓ_2 regularization 0.1 for Stage 2 *rebalanced training*. We apply the same above setting for both JTT and BAM. The best result for JTT occurs when $T = 2$ and $\mu = 4$. The best result for BAM occurs when $T = 2$ and $\mu = 6$. We train for a total of 10 epochs.

CivilComments-WILDS We use batch size 16 for two stages of training. We apply an initial learning rate of 2e-5 for Stage 1 and 1e-5 for Stage 2. We use the SGD optimizer without clipping for Stage 1 and the AdamW optimizer with clipping for Stage 2. We use ℓ_2 regularization 0.01 for Stage 2 *rebalanced training*. We apply the same above setting for

both JTT and BAM. The best result for JTT occurs when $T = 1$ and $\mu = 4$. The best result for BAM occurs when $T = 1$ and $\mu = 4$. We train for a total of 10 epochs.

Colored-MNIST We use batch size 64 and learning rate 1e-4 for two stages of training. We use the stochastic gradient descent (SGD) optimizer with momentum 0.9 throughout the training process. We use a simple convolution network with two convolutional layers followed by a fully connected layer. As a toy dataset, the dataset is only used for visualizations. In Figure 4, we use $\lambda = 20$.

Controlled-Waterbirds We fix $\lambda = 50$ and $T = 100$. For circumstances (1), we test different $\mu \in \{70, 100, 120\}$. For circumstance (2), we test different $\mu \in \{10, 50, 70, 100, 120, 150\}$. For ciscumstance (3), we test different $\mu \in \{10, 50, 70, 100, 120, 150\}$. All other hyperparameters and settings follow what is used in the original Waterbirds dataset.

CIFAR10-MNIST We use the learning rate 1e-5 and batch size 64 for two stages of training. We use the stochastic gradient descent (SGD) optimizer with momentum 0.9 throughout the training process. We use ℓ_2 regularization 1 for Stage 2 *rebalanced training*. We use pretrained ResNet-18 as the training model. We fix $\lambda = 50$, and test different $T \in \{5, 10, 25\}$ and $\mu \in \{5, 10, 20, 50\}$ for circumstances (1) and (2).

C. Further Analysis on Auxiliary Variables

Figure 8 and Figure 9 show the auxiliary variable logits with different T's in Stage 1, and clearly illustrate the trends of changes.

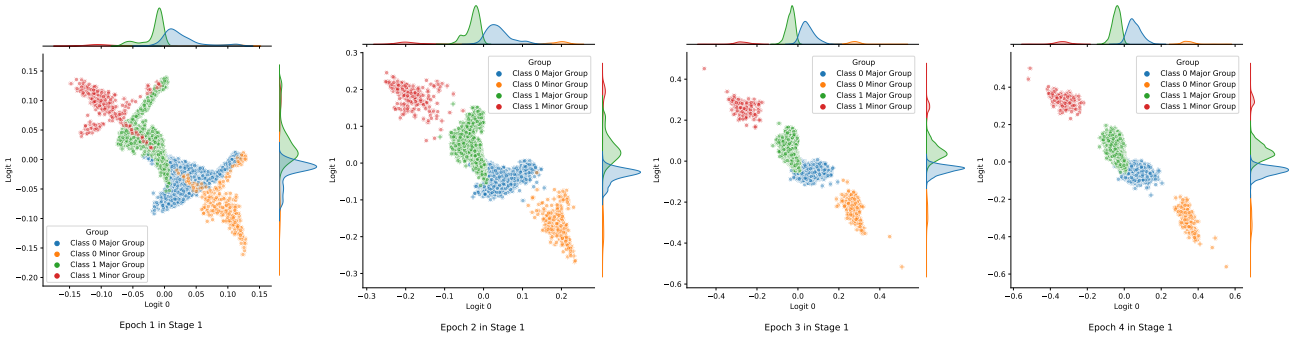

Figure 8. Epoch 1 to 4 in stage 1 on a toy dataset, Colored-MNIST. Logit 0 corresponds to the prediction on class 0, and logit 1 corresponds to class 1.

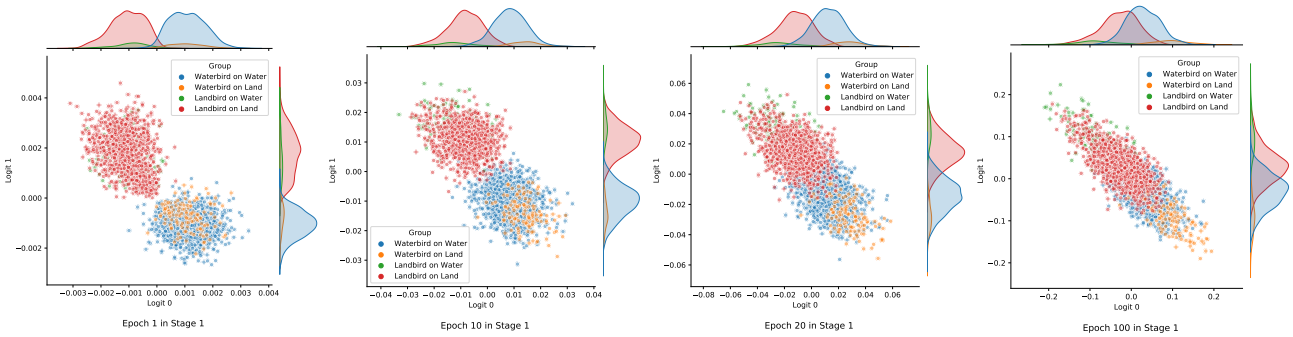

Figure 9. Epoch 1 to 100 in Stage 1 on Waterbirds. Logit 0 corresponds to the prediction on the waterbird class, and logit 1 corresponds to landbird. The group sizes are 1800, 200, 200, 1800 in order.

D. Class Difference Plots and Explanations

We conducted additional experiments on two datasets to verify the robustness of BAM by changing the relative group sizes and class sizes: Controlled-Waterbirds and CIFAR10-MNIST. Considering the circumstances listed in Appendix A and

following the training details in Appendix B, we conduct the experiments, and randomly show some of the ClassDiff plots in Figure 14.

To avoid charry picking, we first classify each experiment into one of the settings in Appendix A. The number of experiments are $12, 18, 6, 36, 36$ for each scenario in order. Then, we sort the following parameters and hyperparameters by the order of importance: dataset parameter values shown in each setting in the dataset details $> \lambda > T > \mu$. Finally, we set random seed to 0 and use the function `NumPy.random.choice` to select 5 independent runs for each circumstance.

The following figures illustrate that both ClassDiff and BAM are robust to varying group and class ratios on the manually generated datasets. In addition, we observed on every single experiment of both these two new established datasets that the class difference is negatively correlated with highest robust test accuracy, regardless of their group sizes and class sizes.

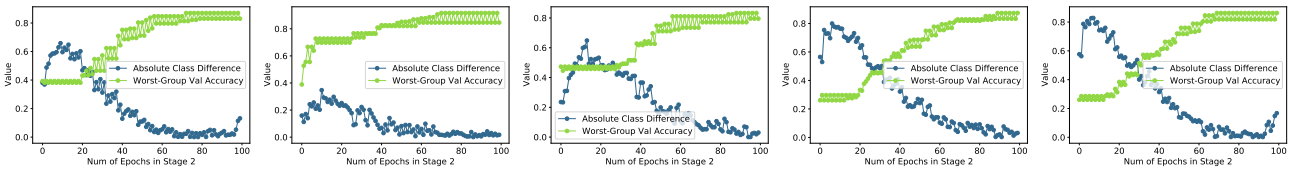

Figure 10. 5 random experiments of the circumstance (1) on the Controlled-Waterbirds dataset.

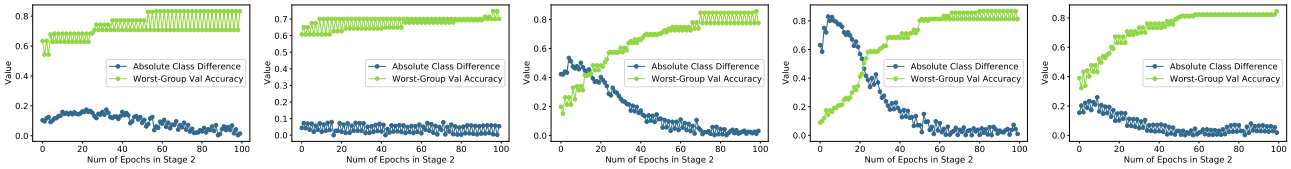

Figure 11. 5 random experiments of the circumstance (2) on the Controlled-Waterbirds dataset.

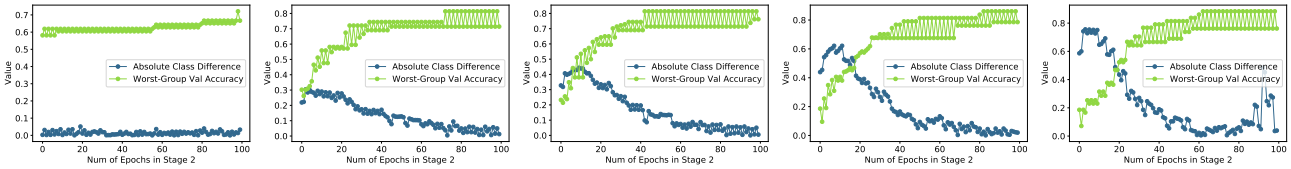

Figure 12. 5 random experiments of the circumstance (3) on the Controlled-Waterbirds dataset.

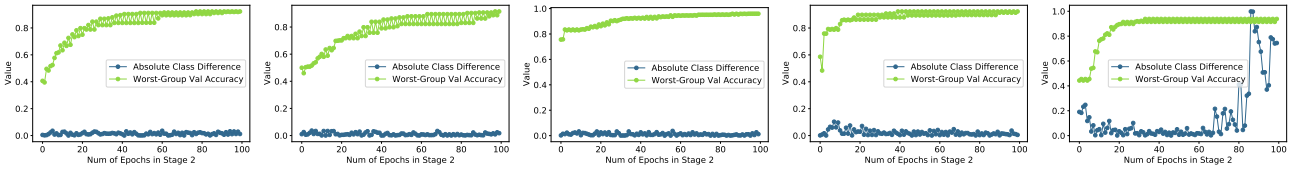

Figure 13. 5 random experiments of the circumstance (1) on the CIFAR10-MNIST dataset.

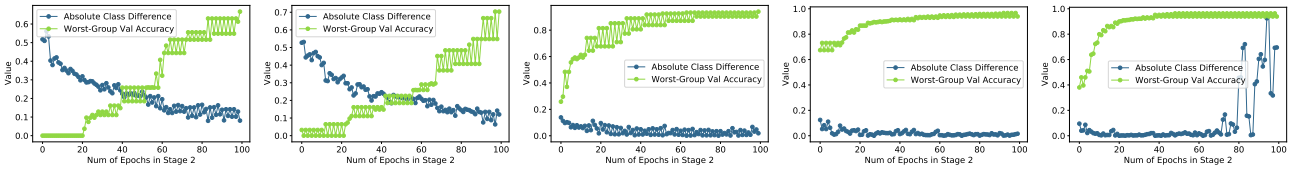

Figure 14. 5 random experiments of the circumstance (2) on the CIFAR10-MNIST dataset.

E. Smaller Validation Set

Table 6 shows the worst-group accuracies on Waterbirds with varying sizes of group-annotated validation set.

Table 6. Worst-group accuracy on Waterbirds with varying size of validation set with group annotations. BAM maintains high worst-group test accuracies even when tuned with very few numbers of group-annotated validation set. However, we note that with a fewer size of validation set with annotations, the performance is actually worse than when tuned for a full-size validation set without any annotations.

Size of Annotated Validation Set	100%	20%	10%	5%
Worst-Group Acc.(%)	89.8	89.1	88.4	86.2

F. Supplementary Figures

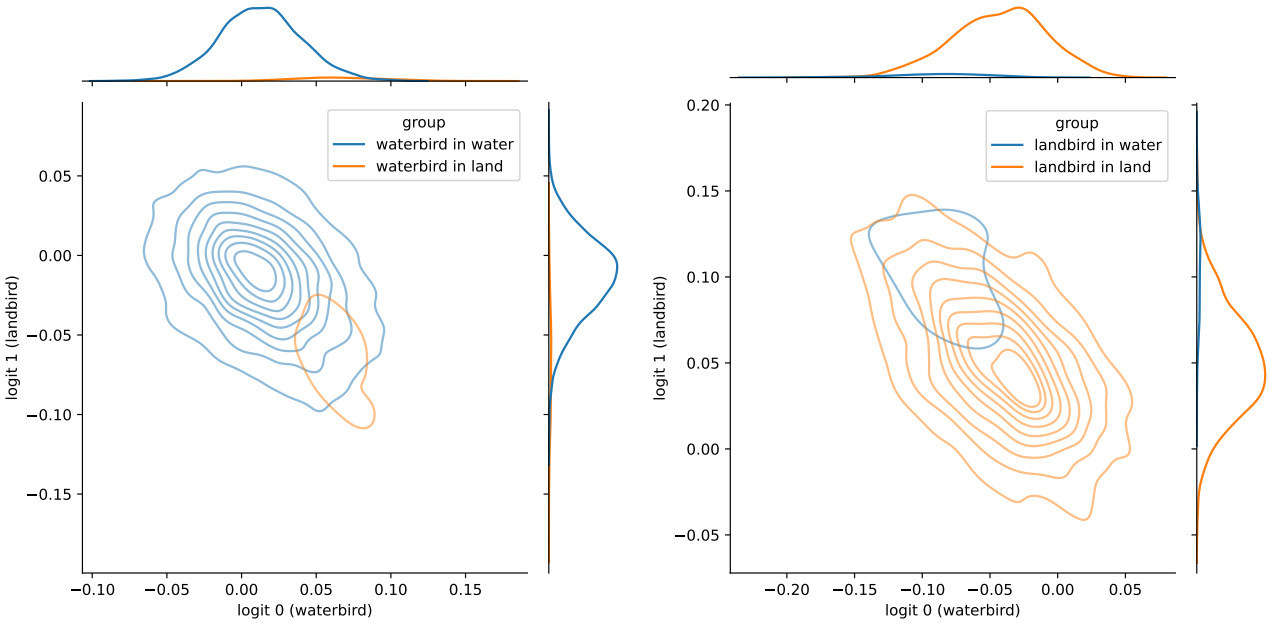

Figure 15. Corresponding KDE plot of Figure 2.